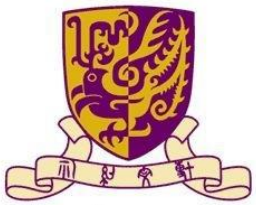


香港中文大學（深圳）
The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice

Jingbang Chen
School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen

About instructor (Jingbang Chen)

- Background
  - Undergrad: 2017-2020, ZJU
  - MS: 2020-2022, Gatech
  - PhD: 2023-2025, Waterloo
  - Research Assistant Professor: 2025-Present, CUHK-SZ
- Contact
  - Email: chenjb@cuhk.edu.cn
  - Website: https://chenjb1997.github.io/
  - Office: Room 517a, Dao Yuan Building
- Research
  - General area: Provably efficient algorithms and data structures, with a focus on graphs.
- Office hour
  - 1 – 2 hours right after the class.
    - Let me know whenever you want to talk at some other time.

Something more about me...

International Grandmaster

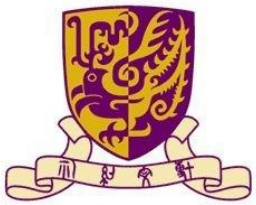
chenjb

Jingbang Chen, Guangzhou, China
From Universal Cup

Contest rating: **2705** (max. **international grandmaster, 2735**)

Awards

- Champion, 42nd International Collegiate Programming Contest Qingdao Regional Contest, 2017
- Champion, 45th International Collegiate Programming Contest Southeast USA Regional, 2021
- Champion, 15th Zhejiang Province Collegiate Programming Competition, 2018
- 3rd Place, 3rd China Collegiate Programming Contest Jilin Regional Contest, 2018
- 3rd Place, 45th International Collegiate Programming Contest North America Championship, 2021
- 56th Place, 42nd International Collegiate Programming Contest World Finals, 2018
- 13th Place, 46th International Collegiate Programming Contest World Finals, 2024
- Gold Medal, 42nd International Collegiate Programming Contest Beijing Regional Contest, 2017
- Gold Medal, 42nd International Collegiate Programming Contest East-Continent League Finals, 2017
- Gold Medal, 43rd International Collegiate Programming Contest Xuzhou Regional Contest, 2018
- Gold Medal, 44th International Collegiate Programming Contest Xuzhou Regional Contest, 2019
- Gold Medal, 44th International Collegiate Programming Contest Shenyang Regional Contest, 2019
- Gold Medal, 44th International Collegiate Programming Contest East-Continent Finals, 2019
- Gold Medal, 2nd China Collegiate Programming Contest Harbin Regional Contest, 2017
- Gold Medal, 2nd China Collegiate Programming Contest Finals, 2017
- Gold Medal, 4th China Collegiate Programming Contest Qinhuangdao Regional Contest, 2019
- Bronze Medal, National Olympiad of Informatics Finals, 2015



香港中文大學（深圳）
The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice Lecture 1: Overview

Jingbang Chen
School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen

Teaching plan & prerequisite courses

- Teaching time and location
  - Time: 13:30 ~ 16:30 on Wednesday
  - Location: TXC

A break of 5-10 mins

- Three prerequisite courses
  - (1) Python, or C/C++, or Java
  - (2) Discrete Mathematics
  - (3) Data Structures

- Special notices
  - This course involves some basics of high-level programming languages for illustrating key concepts and algorithms
  - Students without learning any high-level programming language are not suggested to select this course!

TA Information

- TA: help you with coursework and technical issues; mark your assignments and exams

  - Mr. Chengxuan Pei
    - chengxuanpei@link.cuhk.edu.cn
    - Office Hour: Monday 10:00-12:00
  - Mr. Yumao Xie
    - yumaoxie@link.cuhk.edu.cn
    - Office Hour: TBD

群聊: 2025Spring-CSC5003-Group

该二维码7天内(1月14日前)有效，重新进入将更新

Outline

- Introduction
  - Why/How to take this course?
  - Tentative teaching plan
- Some warm-up coding exercises!!!

What we have learnt from CSC5001

Lecture(s)	Teaching Content
3	Divide-and-conquer
5 - 6	Dynamic Programming 1
7 - 8	Dynamic Programming 2
9	Greedy Algorithm 1
10	Greedy Algorithm 2
11	Union-Find Data Structure
12	Binary Indexed Tree
13 - 14	Segment Tree 1
15 - 16	Segment Tree 2
17	Breadth/Depth-First-Search
18	Shortest Path
19	Minimum Spanning Tree
20	LCA & RMQ
21 – 22	Ad hoc Problems

What weapons do we have now?

- Algorithm design paradigms:
  - Divide-and-conquer
  - Dynamic Programming
  - Greedy
  - Ad hoc
- Data Structures:
  - Binary Indexed Tree
  - Segment Tree
  - Union-find Data Structure
- Graph Algorithms:
  - BFS/DFS
  - Shortest Path: Dijkstra, Bellman-Ford, Floyd
  - Minimum Spanning Tree: Prim, Kruskal
  - LCA & RMQ: Binary Lifting, Sparse Table

Course Goal

- How can we apply these tools to actual algorithmic problem solving?
- How can we code the algorithm fast and precise?
- How can we learn more algorithms by ourselves?

→ This is why you are here.

→ We help you reach a breakthrough in algorithmic problems solving (coding) under any scenarios, such as your future job interviews.

→ Furthermore, we help you get used to the era of AI, staying competitive because you always know more and can do more!

Tentative Teaching Plan

Week	Content/ topic/ activity
1	Course Overview
2	Basic Data Structures: Iterative and Recursion, Standard template library; Graphs and trees
3	Basic Data Structures: Shortest path; Topsort;
4	Basic Data Structures: String processing;
5	Basic Data Structures: Search; Hash-table;
6	Algorithm Design Paradigms: Adhoc problems;
7	Mid-term Exam
8	Algorithm Design Paradigms: Dynamic Programming;
9	Algorithm Design Paradigms: Dynamic Programming;
10	Algorithm Design Paradigms: Greedy, Divide-and-Conquer;
11	Common Interview Questions: Binary series (search, lifting, RMQ)
12	Common Interview Questions: Sliding windows; Sweep Line;
13	Common Interview Questions: Monotonic stack and queue;
14	Common Interview Questions: Small-to-Large Technique;

*The above table may change as the instructor sees fit.

Course assessment

Homework 29% + Midterm exam 30% + Final exam 40% + CTE 1%

- We will be covering ~100 coding problems (range from very easy to hard) all over the semester.
- To get 30% (including CTE 1%) from homework, you should solve 60 problems among them.
  - If you manage to solve it during the class (and present in the class), each problem can be considered as 1.5 problems.
- Midterm: I will select 20 problems one week before and will test 3 problems (30 points each) from them + 1 new problem (10 points).
- Final: : I will select 30 problems one week before and will test 4 problems (20 points each) from them + 1 new problem (20 points).

- Students can use Python/C/C++/Java to answer the questions in Online Judge (OJ) system.
- Some problems may have partials, but getting full correct (AC) gives you full score in this problem.

How to do well in this course?

- Common suggestions
  - Attend the lectures physically
  - Gain more problems credit by solving during the class!
  - Slides will be uploaded to BB/wechat group **after the class**
  - Use examples to facilitate learning
  - Algorithms are not just reading materials; you should write more codes!!!

- Special suggestions
  - If you feel difficult,
    - Try to focus on the content of slides and keep asking questions
  - If you feel easy,
    - Read more details (e.g., theoretical analysis) in the textbook
    - Use the learned techniques to solve ICPC problems

Course policy

- Your work MUST be your own
  - Cheating is against "fair-play" and will not be tolerated under any circumstances

- Assignments
  - Penalty will be imposed on copying assignments; minimum penalty is zero mark for the assignments
  - All coding problems are due 1 day before the exam.
    - Late policy: no more add-on score after the deadline.

Examples of code plagiarism

← 返回 | 代码对比

[redacted].cpp 与 1[redacted].cpp 相似度为 100%

```cpp
#include<bits/stdc++.h>
using namespace std;
int n,a[100100],r[100100];
long long ans=0;
void mergesort(int s,int t)
{
 if(s==t) return ;
 int mid=(s+t)/2;
 mergesort(s,mid);
 mergesort(mid+1,t);
 int i=s,j=mid+1,k=s;
 while(i<=mid&&j<=t)
 {
  if(a[i]<a[j]) r[k++]=a[i++];
  else r[k++]=a[j++],ans+=mid-i+1;
 }
 while(i<=mid) r[k++]=a[i++];
 while(j<=t) r[k++]=a[j++];
 for(int i=s;i<=t;++i)
 a[i]=r[i];
}

int main()
{
 scanf("%d",&n);
 for(int i=1;i<=n;++i)
 scanf("%d",&a[i]);
 mergesort(1,n);
 printf("%lld\n",ans);
```

```cpp
#include<bits/stdc++.h>
using namespace std;
int n,a[500500],r[500500];
long long ans=0;
void msort(int s,int t)
{
        if(s==t) return ;
        int mid=(s+t)/2;
        msort(s,mid);msort(mid+1,t);
        int i=s,j=mid+1,k=s;
        while(i<=mid&&j<=t)
        {
                if(a[i]<a[j]) r[k++]=a[i++];
                else r[k++]=a[j++],ans+=mid-i+1;
        }
        while(i<=mid) r[k++]=a[i++];
        while(j<=t) r[k++]=a[j++];
        for(int i=s;i<=t;++i)
        a[i]=r[i];
}
int main()
{
        scanf("%d",&n);
        for(int i=1;i<=n;++i)
        scanf("%d",&a[i]);
        msort(1,n);
        printf("%lld\n",ans);
}
```

Examples of code plagiarism

Sample1.cpp 与 Sample2.cpp 相似度为 100%

```
#include<iostream>
using namespace std;

void foo1(){
// an easy function
  int a = 10;
  cout<<a;
}

void foo2(){
/* I
   am
   comments*/
  int b = 20;
  cout<<b;
}

int main(){
  foo1();
  foo2();
}
```

```
#include<iostream>
using namespace std;

void fun2(){
  int c=20; cout<<c;
}

void fun1(){
  int d=10; cout<<d;
}

int main(){
  fun1();
  fun2();
}
```

Changing the variable and function names, and re-organizing function orders are typical kinds of code plagiarism… We will use a software tool to detect code plagiarism automatically!

Don't send your source codes to others, which may make you be the one who is copied from! (抄和被抄都会有处罚，默认就是0分)

Content in the class.

- Teach new algorithms/review known algorithms if there is any.
  - A few problems as showcases
- Discuss problems from real-world sources (interview, programming competitions)
- Sufficient time for try completing some of them during the class.
  - I will explicitly specify on problems that we should try together.
  - If there is more time, try doing some homework problems to gain more credits!
- Everyone come to me/TA to specify on extra credits that you have gained.

Some problems for warm-up!

Vjudge Registration

- Please register on https://vjudge.net/ and the account should be in the following format:

- Spring2026 + _ + Student_id

- Example: Spring2026_100012925

Chorus

- There are n students standing in a line. The music teacher wants to ask $n-k$ of them to step out of the line, so that the remaining $k$students can form a choir formation.
- *A choir formation* is defined as follows: label the $k$remaining students from left to right as $1,2,\ldots,k$, and let their heights be $t_1, t_2, \ldots, t_k$ .Their heights must satisfy

$$t_1 < \cdots < t_i > t_{i+1} > \cdots > t_k \ (1 \le i \le k).$$

- Your task is: given the heights of all n students, compute the **minimum number of students that need to step out** so that the remaining students can form a choir formation.

- N<=100, 130<=t_i<=230

Words Counting

- Given a word and a sentence with spaces, find the first occurrence and the number of occurrences (lower-case and upper-case are equivalent)

- Word length <= 10
- Sentence length <= 10^6

Solution

- Just do it!!!

Solution

- Let f[i] to be the longest increasing subsequence that ends with i, g[i] be the longest decreasing subsequence that starts with i.
- Compute f[i]/g[i] with the standard LIS DP
- The answer will be max(f[i]+g[i]-1).

Fast and Fat from CSC5001

You're participating in a team competition of trail running. There are n members in your team where v_i is the speed of the i-th member and w_i is his/her weight.

The competition allows each team member to move alone or carry another team member on their back. When member i carries member j, if member i's weight is greater than or equal to member j's weight, member i's speed remains unchanged at v_i. However, if member i's weight is less than member j's weight, member i's speed will decrease by the difference of their weight and becomes $v_i - (w_j - w_i)$. If member i's speed will become negative, then member i is not able to carry member j. Each member can only carry at most one other member. If a member is being carried, he/she cannot carry another member at the same time.

For all members not being carried, the speed of the slowest member is the speed of the whole team. Find the maximum possible speed of the whole team.

Example

standard input	
2 5 10 5 1 102 10 100 7 4 9 50 2 1 100 10 1	8 1

Note

The optimal strategy for the sample test case is shown as follows:

- Let member 1 carry member 4. As $w_1 > w_4$, member 1's speed remains unchanged, which is still 10.
- Let member 3 carry member 2. As $w_3 < w_2$, member 3's speed will decrease by $w_2 - w_3 = 2$ and becomes $10 - 2 = 8$.
- Member 5 shall move alone. His/Her speed is 9.

So the answer is 8.

Solution

- Binary search the speed is at least X.
- Then for any person with at least speed X, then he can carry a person with v-x+w weight.
- If a person with speed less than X, then he must be carried.

Solution

- This is a classic problem:
  - Given p person and q jobs, the i-th person has an ability level a[i], the j-th job has a difficulty b[j], we can assign job j to person i iff he has no other job and a[i]>=b[j].

- Sort and pick the q persons with highest ability, assign the i-th hardest to the i-th best person.